

# Rapport J13

## Fonctions texte et nettoyage/préparation de données PostgreSQL

|                      |                                                                |
|----------------------|----------------------------------------------------------------|
| Plan                 | Plan d'exécution SQL/Data — 16 semaines                        |
| Semaine              | 02 — J13                                                       |
| Date d'exécution     | 28 septembre 2026                                              |
| Base                 | sql_training                                                   |
| Table                | raw.training_events                                            |
| PostgreSQL           | 17.5                                                           |
| Durée prévue         | 75–90 min                                                      |
| Durée réelle retenue | 75 min                                                         |
| Écart vs 90 min      | –15 min                                                        |
| Statut               | SQL et validation terminés; documentation après arrêt du timer |

### 1. Objectifs

Maîtriser les principales fonctions PostgreSQL de transformation et de préparation de texte sans modifier les données brutes : ``UPPER()``, ``LOWER()``, ``LENGTH()``, ``REPLACE()``, ``INITCAP()``, ``TRIM()``, concaténation ``||`` et ``COALESCE()``. Combiner ces fonctions avec les acquis précédents (``IN``, ``CASE``, ``ORDER BY``, ``LIMIT``) afin de produire un extrait métier lisible et reproductible.

### 2. État initial et validation de l'environnement

Git était propre et synchronisé avec ``origin/main``. Le dernier commit de référence était ``3541b8a`` — ``docs: complete J12 SQL DISTINCT expressions and CASE training``.

La connexion à ``sql_training`` a été confirmée. Le dataset de référence contenait :

- 15 lignes ;
- 4 ``source_system`` distincts ;
- 6 ``event_type`` distincts ;
- ``event_id`` minimum = 1 ;
- ``event_id`` maximum = 18.

Aucune modification destructive du dataset n'a été effectuée pendant J13.

### 3. Notions et résultats

#### 3.1 Observation des données textuelles

Les identifiants techniques observés incluent ``manual_psql``, ``microgrid_poc``, ``sensor_gateway``, ``tomra_training``, ainsi que ``training_test``, ``sensor_reading``, ``machine_alarm``, ``temperature``, ``current`` et ``voltage``.

#### 3.2 UPPER() et LOWER()

``UPPER(source_system)`` produit une représentation en majuscules et ``LOWER(source_system)`` une représentation en minuscules, sans modifier la donnée stockée. Les données sources étant déjà en minuscules, ``LOWER()`` ne change visuellement pas leur contenu.

#### 3.3 LENGTH()

Résultats ``source_system`` :

- ``sensor_gateway`` : 14 ;
- ``tomra_training`` : 14 ;
- ``microgrid_poc`` : 13 ;
- ``manual_psql`` : 11.

Résultats ``event_type`` :

- `sensor\_reading` : 14 ;
- `machine\_alarm` : 13 ;
- `training\_test` : 13 ;
- `temperature` : 11 ;
- `current` : 7 ;
- `voltage` : 7.

### 3.4 REPLACE() et INITCAP()

`REPLACE(..., '\_', ' ')` transforme les identifiants techniques en libellés espacés. L'imbrication `INITCAP(REPLACE(...))` produit ensuite des libellés de présentation comme `Microgrid Poc`, `Sensor Gateway` et `Sensor Reading`.

Point de compréhension : l'ordre d'application des fonctions imbriquées est important. Une transformation intérieure est évaluée avant la transformation extérieure.

### 3.5 TRIM()

Le test contrôlé a montré :

- longueur avant nettoyage : 17 ;
- longueur après `TRIM()` : 13.

Cela démontre la suppression des espaces périphériques sans toucher au contenu utile.

### 3.6 Concaténation

L'opérateur `||` a permis de construire des descripteurs métier. Exemple :

`microgrid\_poc` + `sensor\_reading` → `Microgrid Poc - Sensor Reading`.

### 3.7 COALESCE()

Sur le dataset courant, toutes les valeurs étant renseignées, `COALESCE(event\_value, 0)` conserve les valeurs existantes. Le test contrôlé a confirmé :

- `COALESCE(NULL::numeric, 0)` → 0 ;
- `COALESCE(125.50::numeric, 0)` → 125.50.

Règle métier : remplacer `NULL` par zéro n'est pertinent que lorsque zéro représente réellement la valeur de substitution correcte.

## 4. Exercice guidé J13-10

Objectif : produire `event\_id`, `source\_system`, `source\_label`, `event\_type`, `event\_label`, `event\_value` pour `microgrid\_poc` et `sensor\_gateway`, triés par `event\_value` DESC, `event\_id` ASC.

Prédictions :

- nombre de lignes : 11 ;
- premier `event\_id` : 15 ;
- premier `source\_label` : `Microgrid Poc`.

Résultats : les trois prédictions étaient exactes. La requête a retourné 11 lignes et a correctement produit les libellés métier.

## 5. Challenge autonome J13-11

Objectif : produire `event\_id`, `event\_descriptor`, `event\_value`, `priority`, uniquement pour `microgrid\_poc` et `sensor\_gateway`, triés par valeur décroissante puis ID croissant, avec `LIMIT 8`.

Règles de priorité :

- `NULL` → `UNKNOWN` ;
- `>= 200` → `CRITICAL` ;
- `>= 25` → `REVIEW` ;
- sinon → `NORMAL`.

Prédictions :

- 8 lignes ;
- premier `event\_id` = 15 ;

- premier descripteur = `Microgrid Poc - Sensor Reading` ;
- première priorité = `CRITICAL`.

Résultats : toutes les prédictions étaient exactes. Les cinq premières lignes étaient classées `CRITICAL`, suivies des lignes `REVIEW` attendues.

## 6. Script reproductible et validation

Script créé :

```
`documentation/semaine-02/jour-13/code/J13_text_functions_cleaning.sql`
```

Le script rassemble :

1. validation du dataset ;
2. observation des textes bruts ;
3. `UPPER()` / `LOWER()` ;
4. `LENGTH()` ;
5. `REPLACE()` ;
6. `INITCAP()` ;
7. `TRIM()` ;
8. concaténation ;
9. `COALESCE()` ;
10. exercice guidé ;
11. challenge autonome ;
12. contrôle final d'intégrité.

Commande de validation :

```
psql -U postgres -d sql_training -f ".\documentation\semaine-02\jour-13\code\J13_text_functions_cleaning.sql"
```

La validation complète s'est exécutée sans erreur visible. Les contrôles finaux ont confirmé :

- `total\_rows` = 15 ;
- `min\_event\_id` = 1 ;
- `max\_event\_id` = 18 ;
- `total\_value` = 1583.40.

## 7. Erreurs, observations et résolutions

Aucune erreur SQL bloquante n'a été observée. Le test `TRIM()` a été stabilisé dans le script reproductible avec un nombre explicite d'espaces afin de conserver une démonstration cohérente `17 → 13`. L'avertissement Windows concernant les pages de codes 850/1252 reste connu et n'a pas empêché l'exécution.

## 8. Captures à conserver

- `J13-01\_environment-dataset-validation.png`
- `J13-02\_raw-text-business-data.png`
- `J13-03\_upper-lower-functions.png`
- `J13-04\_text-length-analysis.png`
- `J13-05\_replace-technical-labels.png`
- `J13-06\_business-readable-labels.png`
- `J13-07\_trim-whitespace-cleaning.png`
- `J13-08\_text-concatenation-business-label.png`
- `J13-09\_coalesce-null-handling.png`
- `J13-10\_guided-text-transformation-exercise.png`
- `J13-11\_autonomous-text-business-challenge.png`

- `J13-12\_text-functions-script-validation-1.png`
- `J13-12\_text-functions-script-validation-2.png`
- `J13-12\_text-functions-script-validation-3.png`

## 9. KPI de progression

- Fonctions texte fondamentales exécutées : 8/8.
- Prédications J13-10 correctes : 3/3.
- Prédications J13-11 correctes : 4/4.
- Challenge autonome : réussi.
- Validation `psql -f` : réussie.
- Erreurs SQL bloquantes : 0.
- Intégrité du dataset : conservée (`15 / 1 / 18 / 1583.40`).
- Capacité démontrée : transformer des identifiants techniques en libellés métier et combiner préparation de texte, logique conditionnelle, filtrage, tri et Top-N.

## 10. Git / versionnement

Le travail Git est volontairement effectué après l'arrêt du timer et ne doit pas modifier la durée réelle de J13.

Commit cible :

`docs: complete J13 PostgreSQL text functions and data cleaning training`

Avant commit : contrôler `git status`, le contenu staged et les fichiers de `documentation/semaine-02/jour-13/`. Après commit : pousser sur `origin main`, puis confirmer `HEAD -> main` et `origin/main`.

## 11. Durée

- Durée prévue : 75–90 min.
- Timer de référence lancé sur 90 min.
- Timer arrêté avec 15 min restantes.
- Durée réelle retenue : **\*\*75 min\*\***.
- Écart par rapport aux 90 min : **\*\*–15 min\*\***.
- La documentation et les opérations Git postérieures à l'arrêt ne sont pas ajoutées à cette durée.

## 12. Conclusion

J13 marque le passage d'une manipulation de valeurs techniques à une préparation de données orientée usage métier. Les transformations ont été réalisées en lecture seule et intégrées dans un script reproductible. Le challenge autonome montre une capacité à composer plusieurs notions SQL acquises depuis J08 dans une seule requête analytique.

## 13. Prochaines étapes

J14 sera consacré à la consolidation de la Semaine 02 : filtrage, agrégations, `GROUP BY`, `HAVING`, `DISTINCT`, `CASE`, fonctions texte et challenge métier intégré. J13 doit toutefois être versionné et techniquement clôturé avant la clôture définitive de la semaine.

## 14. Potentiel de réutilisation pédagogique

J13 peut être transformé en :

- tutoriel « Nettoyer et rendre lisibles des données techniques avec PostgreSQL » ;
- vidéo éducative sur `REPLACE`, `INITCAP`, `TRIM`, concaténation et `COALESCE` ;
- laboratoire pratique de préparation de données avant Power BI ;
- exercice de formation sur le passage d'identifiants techniques à une couche de présentation analytique ;
- actif de portfolio démontrant une démarche reproductible et documentée.